

Cidades ESG Inteligentes

Documentação Técnica — Pipeline CI/CD, Containerização e Orquestração

Campo	Informação
Integrante	Danilo Lima Cruz — RM _____
Curso	Análise e Desenvolvimento de Sistemas — FIAP
Fase	Navegando pelo mundo DevOps
Projeto base	API REST Java Spring Boot com tema ESG
Repositório	https://github.com/dlcruzz/cidades-esg-inteligentes

1. Visão geral

A **Cidades ESG Inteligentes** é uma API REST em Java Spring Boot para cadastrar cidades e acompanhar seus indicadores Ambiental, Social e de Governança (0 a 100), com cálculo automático do score ESG. Neste desafio a aplicação foi preparada para um ciclo de vida de produção: cada push na branch main compila, testa, gera uma imagem Docker e faz deploy automatizado em dois ambientes — **staging** e **produção** — com validação automática (smoke test) em cada um.

Método	Rota	Descrição
GET	/	Status e ambiente ativo (staging/production)
GET	/api/cidades	Lista (filtros ?uf= e ?nome=)
GET	/api/cidades/{id}	Busca por id (404 se não existir)
POST / PUT / DELETE	/api/cidades[/]{id}]	Cria, atualiza e remove
GET	/actuator/health	Health check usado por Docker, K8s e pipeline

2. Pipeline CI/CD

Ferramenta: GitHub Actions, arquivo `.github/workflows/ci-cd.yml`.

2.1 Gatilhos

- **push na main:** pipeline completo, até produção.
- **pull_request para main:** apenas build e testes, como gate de qualidade antes do merge.
- **workflow_dispatch:** execução manual pela aba Actions.

2.2 Etapas e lógica

`build > test > docker-build-push > deploy-staging > deploy-production`

Job	O que faz
build	Configura o JDK 17, executa mvn clean package -DskipTests e publica o .jar como artefato.
test	Executa os 11 testes automatizados (JUnit 5, Mockito, MockMvc) com o profile test (H2 em memória). Publica o relatório Surefire e um resumo na página da execução.
docker-build-push	Faz o build da imagem pelo Dockerfile e envia ao GitHub Container Registry (GHCR) com as tags SHA do commit e latest.
deploy-staging	Environment staging. Baixa a imagem do GHCR e roda scripts/deploy.sh staging: Docker Compose sobe API + PostgreSQL com profile staging e o smoke test valida a aplicação.
deploy-production	Só roda se staging passou (needs). Environment production com aprovação manual (required reviewers). Roda scripts/deploy.sh production com o override de produção e o mesmo smoke test.

2.3 Staging e produção

Os dois ambientes usam **a mesma imagem** (tag do commit), mudando apenas configuração — princípio “build once, deploy many”. Cada um roda como um projeto Docker Compose separado (cidades-esg-staging e cidades-esg-production), com rede, volume e porta próprios. Produção aplica o arquivo docker-compose.production.yml: profile production, limites de CPU/memória, restart: always e banco sem porta exposta. Os ambientes são criados no runner do GitHub Actions a cada execução; como o runner é descartado no fim do job, as evidências ficam no resumo da execução e nos artefatos logs-staging e logs-production. Para um servidor permanente, o mesmo script é chamado via SSH.

2.4 Smoke test

O script scripts/smoke-test.sh aguarda /actuator/health retornar UP, confere se o ambiente ativo é o esperado, cria uma cidade (POST), consulta por UF (GET) e valida o retorno 404 para id inexistente. Qualquer falha interrompe o pipeline e impede a promoção para produção.

3. Docker

3.1 Arquitetura

Imagem com **build multi-stage**: o estágio build usa maven:3.9-eclipse-temurin-17 para compilar; o estágio final parte de eclipse-temurin:17-jre-jammy e recebe só o .jar. A aplicação roda com usuário não-root, tem HEALTHCHECK no Actuator e a JVM limitada a 75% da memória do container.

Componente	Detalhe
app	Imagem ghcr.io/dlcruzz/cidades-esg-inteligentes (porta 8080 interna)
db	postgres:16-alpine com healthcheck pg_isready
Rede	esg-network (bridge), isolada por ambiente
Volume	esg-db-data — persistência do PostgreSQL
Variáveis	Arquivo .env (modelo .env.example): profile, porta, credenciais, imagem
Portas	local 8080 · staging 8081 · produção 8082

3.2 Dockerfile

```
# =====
# Dockerfile - Cidades ESG Inteligentes
# Estratégia: build multi-stage para manter a imagem final
# enxuta (sem Maven/JDK completo) e mais segura (usuário
# não-root, apenas JRE em runtime).
# =====

# ----- STAGE 1: build -----
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copia apenas o pom.xml primeiro para aproveitar o cache de camadas
# do Docker: dependências só são baixadas de novo se o pom mudar.
COPY pom.xml .
RUN mvn -B dependency:go-offline

# Copia o restante do código-fonte e gera o artefato .jar
COPY src ./src
RUN mvn -B clean package -DskipTests

# ----- STAGE 2: runtime -----
FROM eclipse-temurin:17-jre-jammy AS runtime

# wget é usado apenas pelo HEALTHCHECK abaixo
RUN apt-get update && apt-get install -y --no-install-recommends wget \
    && rm -rf /var/lib/apt/lists/*

# Usuário não-root por segurança
RUN groupadd -r esg && useradd -r -g esg esgapp

WORKDIR /app

# Copia apenas o .jar gerado no estágio de build (imagem final não tem Maven/JDK)
COPY --from=build /app/target/cidades-esg-inteligentes.jar app.jar

RUN chown -R esgapp:esg /app
USER esgapp

EXPOSE 8080

# Healthcheck usado pelo docker-compose / orquestrador
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:8080/actuator/health || exit 1

# MaxRAMPercentage: a JVM respeita o limite de memória do container
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

3.3 Comandos usados

<code>docker compose --env-file .env up -d --build</code>	# ambiente local
<code>docker compose ps</code>	# status dos containers
<code>bash scripts/deploy.sh staging</code>	# staging -> :8081
<code>bash scripts/deploy.sh production</code>	# produção -> :8082
<code>bash scripts/destroy.sh staging --volumes</code>	# derruba ambiente
<code>docker build -t cidades-esg-inteligentes .</code>	# só a imagem
<code>kubectl apply -f k8s/</code>	# alternativa Kubernetes

3.4 Imagem criada

A cada push na main o pipeline publica

`ghcr.io/dlcruzz/cidades-esg-inteligentes:<sha>` e `:latest` no GitHub Container Registry (aba Packages do repositório).

3.5 Alternativa Kubernetes

A pasta k8s/ traz Deployment + Service da API (2 réplicas, readiness/liveness em /actuator/health, limites de recursos), StatefulSet + Service headless do PostgreSQL com PersistentVolumeClaim e um ConfigMap/Secret de exemplo.

4. Evidências do pipeline

Pipeline completo — todos os jobs concluídos

[inserir print: docs/prints/01-pipeline.png]

Testes automatizados passando

[inserir print: docs/prints/02-testes.png]

Imagem publicada no GitHub Container Registry

[inserir print: docs/prints/03-imagem-ghcr.png]

5. Evidências dos ambientes

Staging: deploy + smoke test aprovado

[inserir print: docs/prints/04-deploy-staging.png]

Produção: aprovação manual do deploy

[inserir print: docs/prints/05-aprovacao-producao.png]

Produção: deploy + smoke test aprovado

[inserir print: docs/prints/06-deploy-producao.png]

Docker Compose local (app + db)

[inserir print: docs/prints/07-docker-compose-local.png]

API respondendo

[inserir print: docs/prints/08-api-funcionando.png]

6. Desafios encontrados e soluções

Desafio	Como foi resolvido
Testes no CI dependiam de um PostgreSQL rodando	Profile test com H2 em memória e dialeto H2 explícito; o pipeline testa sem infraestrutura externa.
Produção não subia em banco novo: ddl-auto=validate exige a tabela já criada	Produção passou a usar update, que cria a tabela no primeiro deploy e nunca apaga dados.
Staging e produção não podiam rodar juntos por causa de container_name fixo	Removido o nome fixo; cada ambiente é um projeto Compose (-p) com rede, volume e porta próprios.
Job de produção referenciava a imagem de um job que não estava em needs (tag vazia)	needs: [docker-build-push, deploy-staging], garantindo a mesma imagem testada em staging.
Deploy era apenas simulado (echo)	Script deploy.sh real com Docker Compose + smoke-test.sh com requisições HTTP; falha bloqueia produção.
Erro escondido pelo tee no log	Shell padrão do workflow com -eo pipefail.
Nome de imagem com maiúsculas é rejeitado pelo GHCR	Nome convertido para minúsculas no pipeline.
Scripts .sh editados no Windows quebram no Linux (CRLF)	.gitattributes força final de linha LF em .sh, .yaml e Dockerfile.
Expor detalhes internos em produção	Profile production sem stacktrace/mensagem nos erros, health sem detalhes e banco sem porta publicada.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☐
Dockerfile funcional	☐
docker-compose.yml ou arquivos Kubernetes	☐
Pipeline com etapas de build, teste e deploy	☐
README.md com instruções e prints	☐
Documentação técnica com evidências (PDF ou PPT)	☐
Deploy realizado nos ambientes staging e produção	☐